INDEX

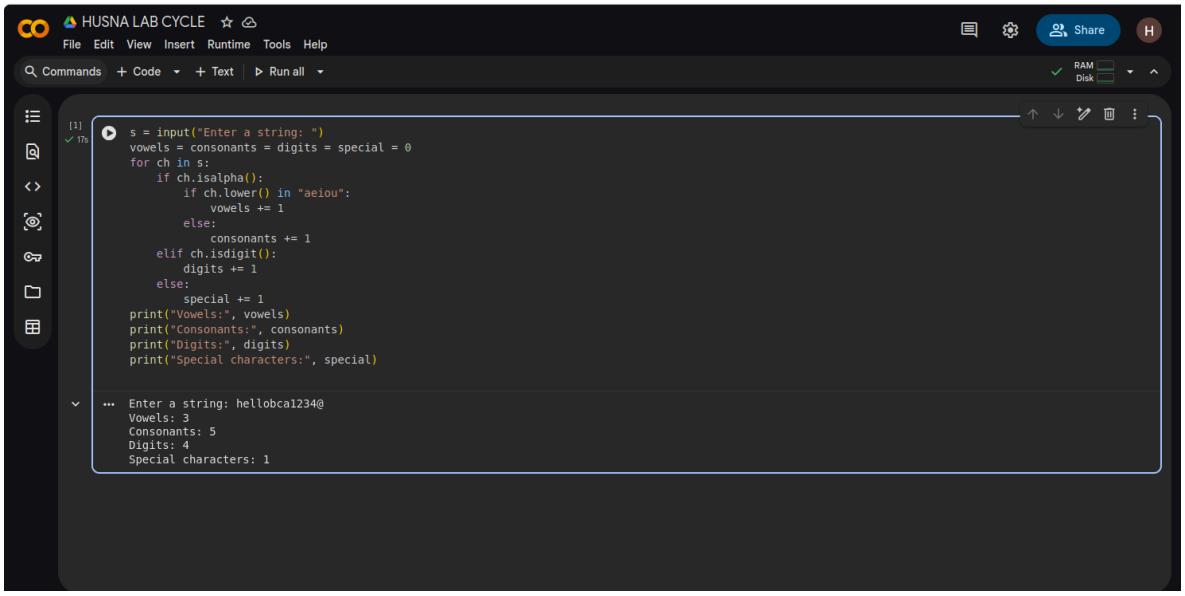
SNO.	CONTENT	PAGE NO.
1	Count vowels, consonants, digits and special characters in a string	
2	Find the largest and second largest elements in a list	
3	Count word frequency using a dictionary	
4	Sort a dictionary by keys and by values	
5	Generate prime numbers using list comprehension	
6	Generate Fibonacci series using loops	
7	Sort strings by length using a lambda function	
8	Sort a dictionary using multiple values and a lambda function	
9	Demonstrate inheritance and polymorphism	
10	Read and write text files using exceptions	
11	Create and analyze a student dataset using Pandas	
12	Create a DataFrame and perform data analysis operations	
13	Perform operations on the student DataFrame	
14	Clean and preprocess a student dataset using Pandas	
15	Plot the mathematical function $y = 3x^2 + 4x$	
16	Create a simple login form using Streamlit	
17	Create a student registration form using Streamlit	
18	Sample program using Git and GitHub	

1) Write a program to count vowels, consonants, digits, and special characters in a string given as input

PROGRAM:

```
s = input("Enter a string: ")
vowels = consonants = digits = special = 0
for ch in s:
    if ch.isalpha():
        if ch.lower() in "aeiou":
            vowels += 1
        else:
            consonants += 1
    elif ch.isdigit():
        digits += 1
    else:
        special += 1
print("Vowels:", vowels)
print("Consonants:", consonants)
print("Digits:", digits)
print("Special characters:", special)
```

OUTPUT:

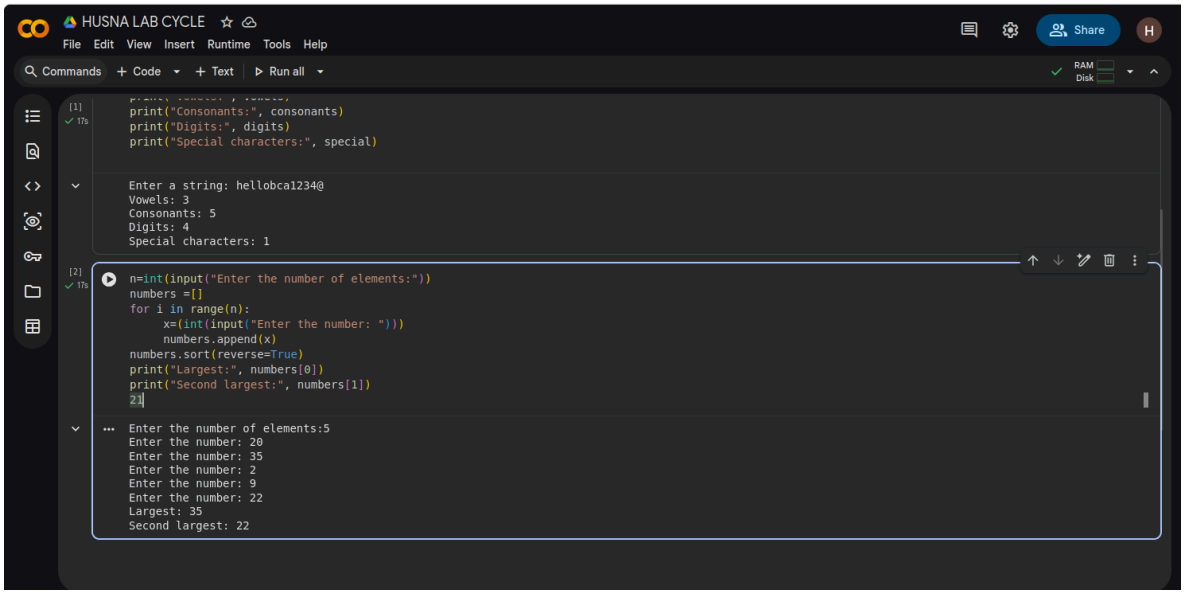


2) Write a program to find the largest and second largest element in a list.

PROGRAM:

```
n=int(input("Enter the number of elements:"))
numbers =[]
for i in range(n):
    x=(int(input("Enter the number: ")))
    numbers.append(x)
numbers.sort(reverse=True)
print("Largest:", numbers[0])
print("Second largest:", numbers[1])
```

OUTPUT:

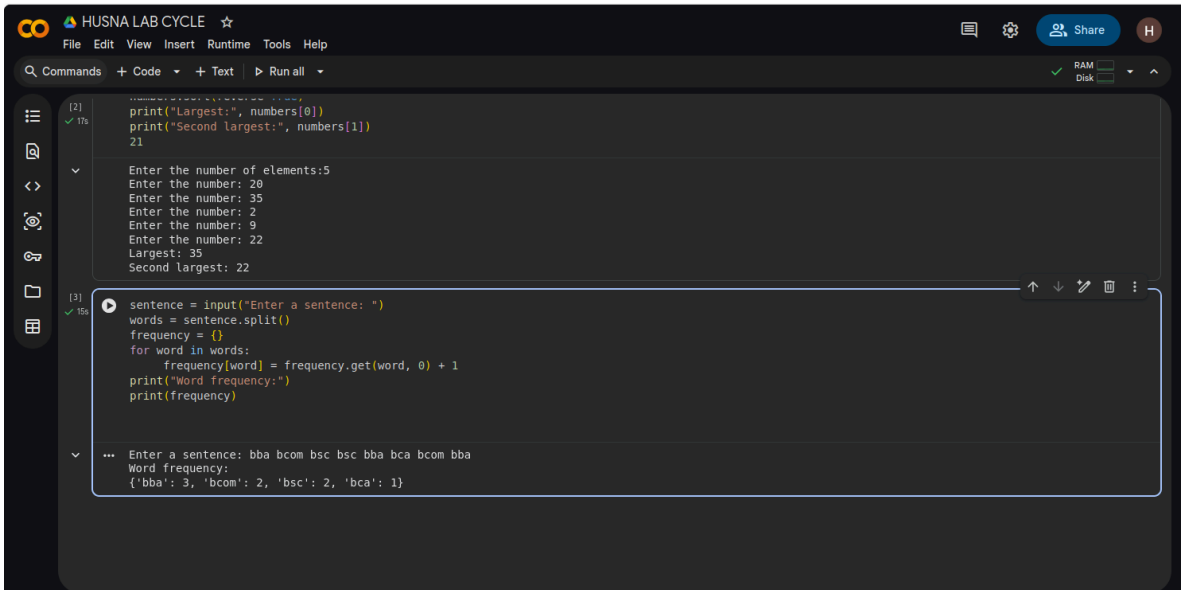


3)Write a program to count word frequency using a dictionary

PROGRAM:

```
sentence = input("Enter a sentence: ")
words = sentence.split()
frequency = {}
for word in words:
    frequency[word] = frequency.get(word, 0) + 1
print("Word frequency:")
print(frequency)
```

OUTPUT:

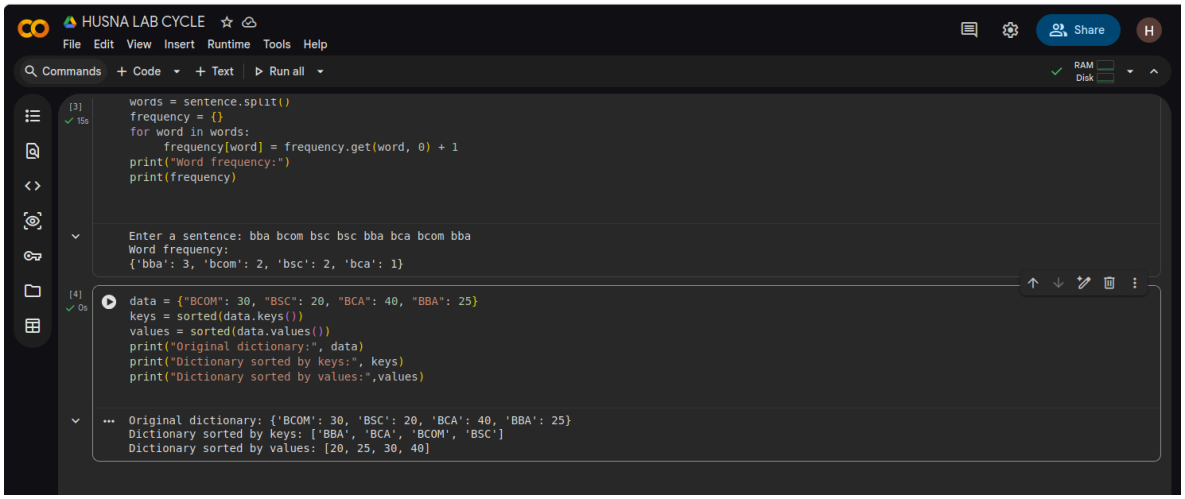


4) Write a program to sort dictionary by keys and then by values.

PROGRAM:

```
data = {"BCOM": 30, "BSC": 20, "BCA": 40, "BBA": 25}
keys = sorted(data.keys())
values = sorted(data.values())
print("Original dictionary:", data)
print("Dictionary sorted by keys:", keys)
print("Dictionary sorted by values:",values)
```

OUTPUT:

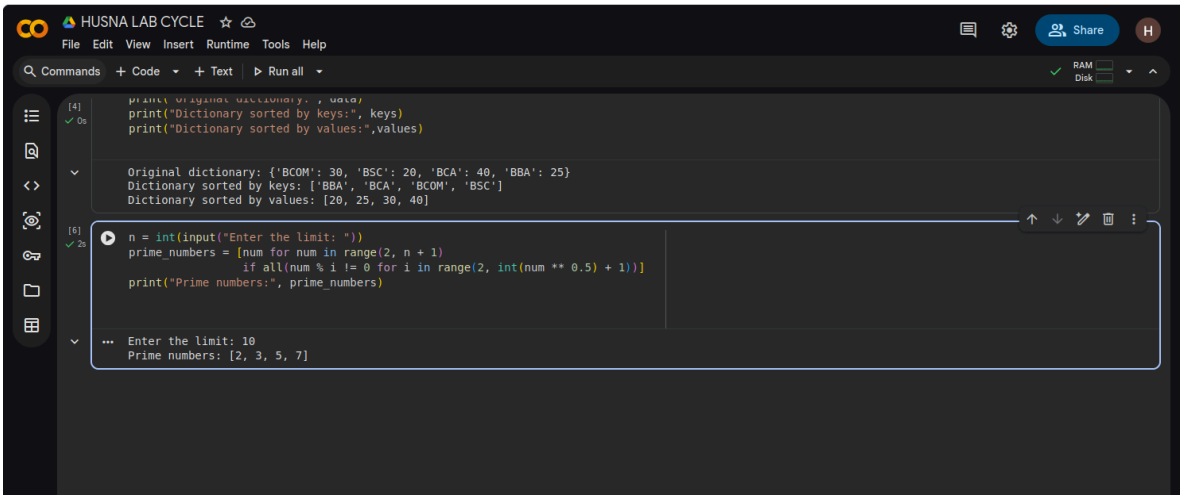


5) Write a program using list comprehension to generate prime numbers.

PROGRAM:

```
n = int(input("Enter the limit: "))
prime_numbers = [num for num in range(2, n + 1)
                 if all(num % i != 0 for i in range(2, int(num ** 0.5) + 1))]
print("Prime numbers:", prime_numbers)
```

OUTPUT:

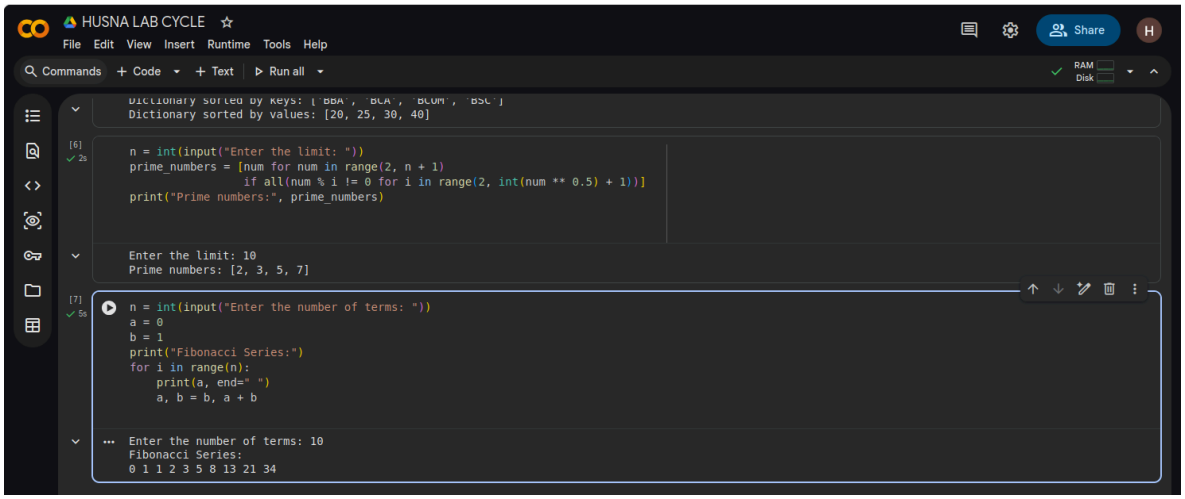


6) Write a program to generate Fibonacci series using loops.

PROGRAM:

```
n = int(input("Enter the number of terms: "))
a = 0
b = 1
print("Fibonacci Series:")
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

OUTPUT:

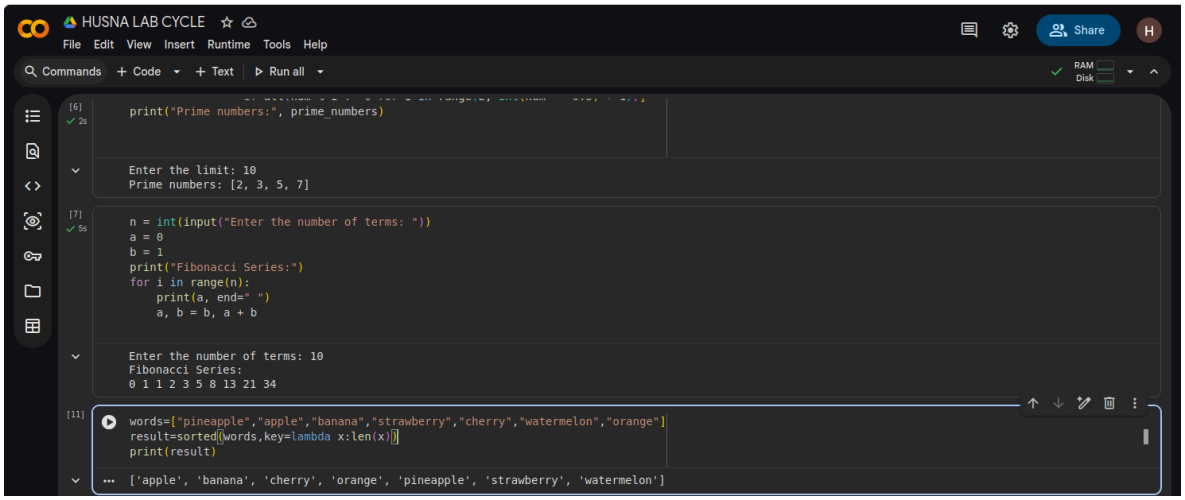


7) Sort strings by length using lambda function.

PROGRAM:

```
words=["pineapple","apple","banana","strawberry","cherry","watermelon","orange"]
result=sorted(words,key=lambda x:len(x))
print(result)
```

OUTPUT:

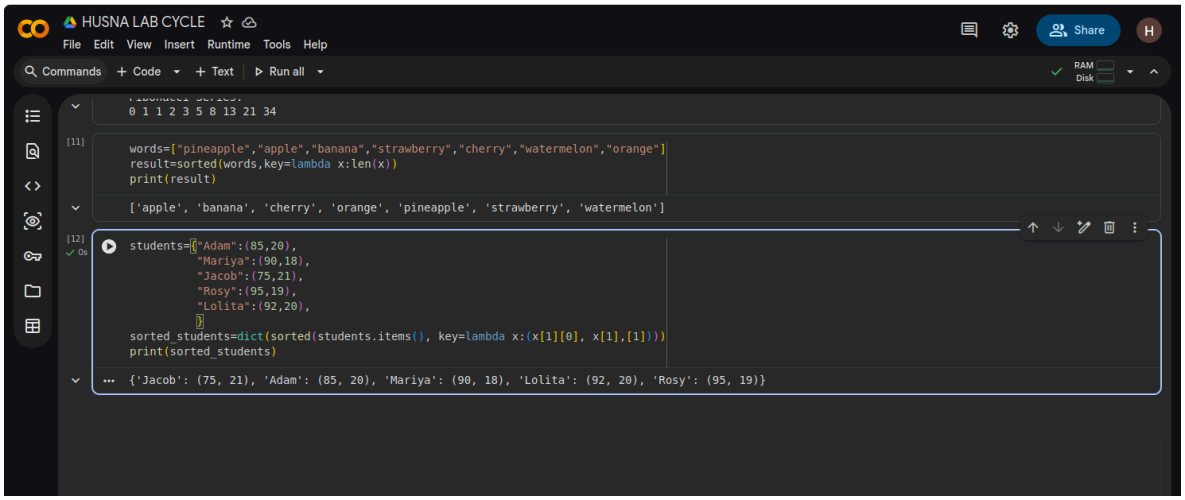


8) Sort dictionary using multiple values and lambda function.

PROGRAM:

```
students={"Adam":(85,20),
        "Mariya":(90,18),
        "Jacob":(75,21),
        "Rosy":(95,19),
        "Lolita":(92,20),
        }
sorted_students=dict(sorted(students.items(), key=lambda x:(x[1][0], x[1],[1])))
print(sorted_students)
```

OUTPUT:



9)

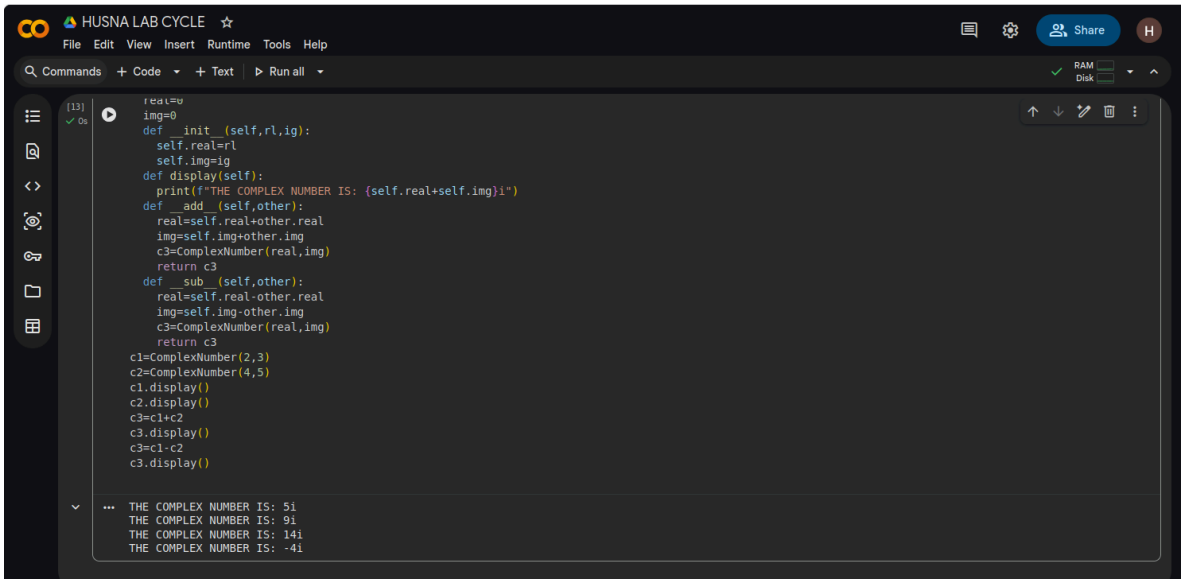
a) Write a program to demonstrate polymorphism using function overloading and operator overloading.
b) Write a program to demonstrate Inheritance

PROGRAM:

a)

```
class ComplexNumber:
  real=0
  img=0
  def __init__(self,rl,ig):
    self.real=rl
    self.img=ig
  def display(self):
    print(f"THE COMPLEX NUMBER IS: {self.real+self.img}i")
  def __add__(self,other):
    real=self.real+other.real
    img=self.img+other.img
    c3=ComplexNumber(real,img)
    return c3
  def __sub__(self,other):
    real=self.real-other.real
    img=self.img-other.img
    c3=ComplexNumber(real,img)
    return c3
c1=ComplexNumber(2,3)
c2=ComplexNumber(4,5)
c1.display()
c2.display()
c3=c1+c2
c3.display()
c3=c1-c2
c3.display()
```

OUTPUT:


HUSNA LAB CYCLE
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
[13]
0s
real=0
img=0
def __init__(self,rl,ig):
self.real=rl
self.img=ig
def display(self):
print(f"THE COMPLEX NUMBER IS: {self.real+self.img}i")
def __add__(self,other):
real=self.real+other.real
img=self.img+other.img
c3=ComplexNumber(real,img)
return c3
def __sub__(self,other):
real=self.real-other.real
img=self.img-other.img
c3=ComplexNumber(real,img)
return c3
c1=ComplexNumber(2,3)
c2=ComplexNumber(4,5)
c1.display()
c2.display()
c3=c1+c2
c3.display()
c3=c1-c2
c3.display()
THE COMPLEX NUMBER IS: 5i
THE COMPLEX NUMBER IS: 9i
THE COMPLEX NUMBER IS: 14i
THE COMPLEX NUMBER IS: -4i

PROGRAM:

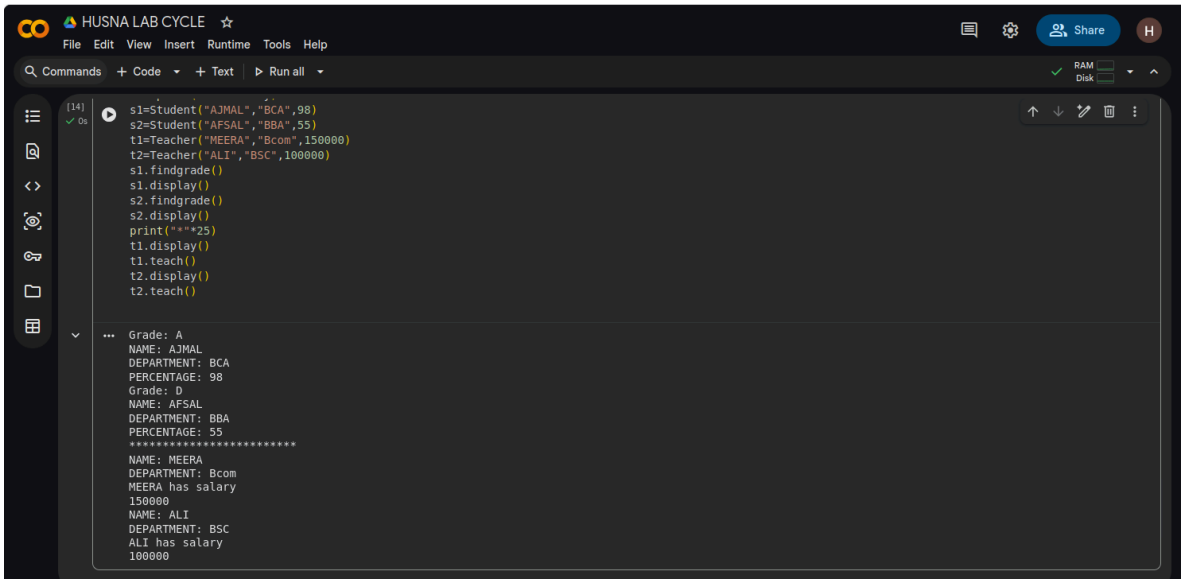
b)

```
class Person:
  def __init__(self, name, department):
    self.name = name
    self.department=department
  def display(self):
    print("NAME:", self.name)
    print("DEPARTMENT:", self.department)

class Student(Person):
    def __init__(self,name,department,perc):
      super().__init__(name,department)
      self.perc=perc
    def findgrade(self):
      if self.perc>=90:
        print("Grade: A")
      elif self.perc>=80:
        print("Grade: B")
      elif self.perc>=70:
        print("Grade: C")
      else:
        print("Grade: D")
    def display(self):
      print("NAME:",self.name)
      print("DEPARTMENT:",self.department)
      print("PERCENTAGE:",self.perc)

class Teacher(Person):
  def __init__(self,name,department,salary):
    super().__init__(name,department)
    self.salary=salary
  def teach(self):
    print(self.name,"has salary")
    print(self.salary)
s1=Student("AJMAL","BCA",98)
s2=Student("AFSAL","BBA",55)
t1=Teacher("MEERA","Bcom",150000)
t2=Teacher("ALI","BSC",100000)
s1.findgrade()
s1.display()
s2.findgrade()
s2.display()
print("*"*25)
t1.display()
t1.teach()
t2.display()
t2.teach()
```

OUTPUT:


HUSNA LAB CYCLE
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
[14]
0s
s1=Student("AJMAL","BCA",98)
s2=Student("AFSAL","BBA",55)
t1=Teacher("MEERA","Bcom",150000)
t2=Teacher("ALI","BSC",100000)
s1.findgrade()
s1.display()
s2.findgrade()
s2.display()
print("*"*25)
t1.display()
t1.teach()
t2.display()
t2.teach()
Grade: A
NAME: AJMAL
DEPARTMENT: BCA
PERCENTAGE: 98
Grade: D
NAME: AFSAL
DEPARTMENT: BBA
PERCENTAGE: 55
****
NAME: MEERA
DEPARTMENT: Bcom
MEERA has salary
150000
NAME: ALI
DEPARTMENT: BSC
ALI has salary
100000

10) Write a program to read and write text files .Use exceptions.

PROGRAM:

```
try:

    file = open("sample.txt", "w")
    file.write("Welcome to Python Programming")
    file.close()

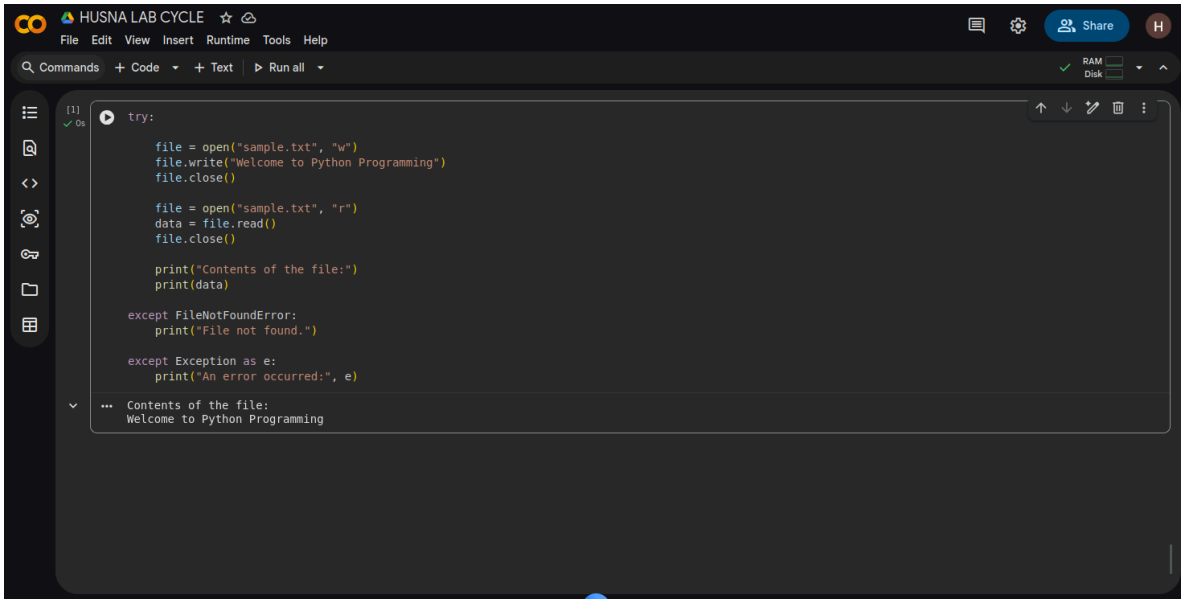

    file = open("sample.txt", "r")
    data = file.read()
    file.close()

    print("Contents of the file:")
    print(data)

except FileNotFoundError:
    print("File not found.")

except Exception as e:
    print("An error occurred:", e)
```

OUTPUT:



11) Write a Python program using Pandas

a. to create and analyze a student dataset containing student name, marks and grade.

b. Use the following data:

data = { "Name": ["Aman", "Diya", "Rahul", "Sneha", "Arjun"],
"Marks": [78, 92, 85, 67, 74],
"Grade": ["B", "A", "A", "C", "B"]}

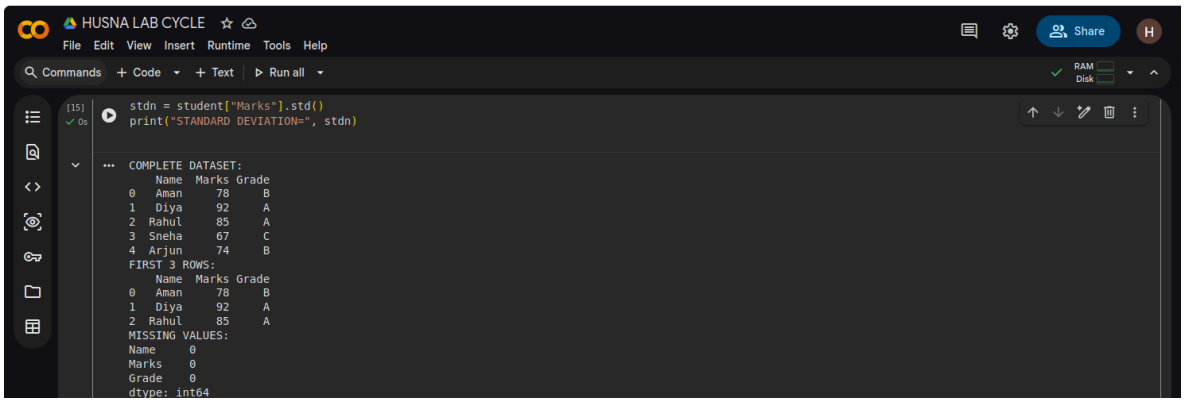
c. Perform the following operations:

i. Create a Pandas DataFrame using the given dataset.
ii. Display the complete dataset.
iii. Display the first 3 rows of the dataset.
iv. Check whether the dataset contains missing values.
v. Create a histogram showing mark distribution.
vi. Find median and standard deviation of marks

PROGRAM:

```
import pandas as pd
import matplotlib.pyplot as plt
data = { "Name": ["Aman", "Diya", "Rahul", "Sneha","Arjun"],
       "Marks": [78, 92, 85, 67, 74],
       "Grade": ["B", "A", "A", "C", "B"]}
student=pd.DataFrame(data)
print("COMPLETE DATASET:")
print(student)
print("FIRST 3 ROWS:")
print(student.head(3))
print("MISSING VALUES:")
print(student.isnull().sum())
plt.figure(figsize=(5, 3))
plt.hist(student["Marks"],
        bins=7,
        color="pink",
        edgecolor="black")
plt.title("MARKS OF STUDENTS")
plt.xlabel("Marks")
plt.ylabel("No.of Students")
plt.show()
median = student["Marks"].median()
print("MEDIAN=", median)
stdn = student["Marks"].std()
print("STANDARD DEVIATION=", stdn)
```

OUTPUT:


HUSNA LAB CYCLE
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
[15] 0s
stdn = student["Marks"].std()
print("STANDARD DEVIATION=", stdn)
COMPLETE DATASET:
Name Marks Grade
0 Aman 78 B
1 Diya 92 A
2 Rahul 85 A
3 Sneha 67 C
4 Arjun 74 B
FIRST 3 ROWS:
Name Marks Grade
0 Aman 78 B
1 Diya 92 A
2 Rahul 85 A
MISSING VALUES:
Name 0
Marks 0
Grade 0
dtype: int64

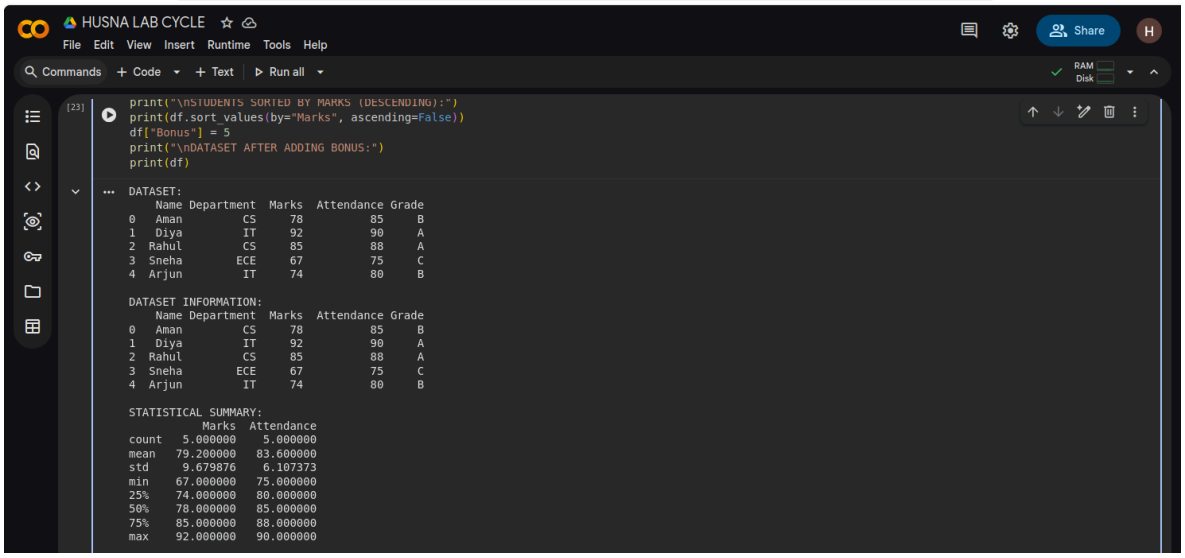

HUSNA LAB CYCLE
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
Marks 0
Grade 0
dtype: int64
MARKS OF STUDENTS
No.of Students
1.0
0.8
0.6
0.4
0.2
0.0
70
75
80
85
90
Marks
MEDIAN= 78.0
STANDARD DEVIATION= 9.67987603226405

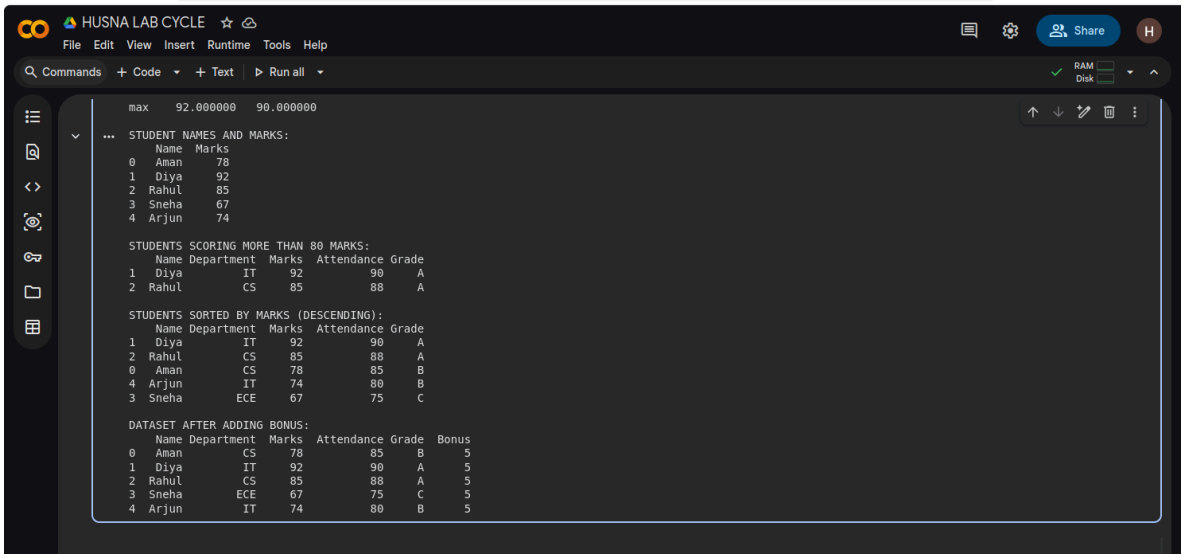
12) Create dataframe from the given
a. data = {"Name": ["Aman", "Diya", "Rahul", "Sneha", Arjun"],
"Department": ["CS", "IT", "CS", "ECE", "IT"],
"Marks": [78, 92, 85, 67, 74],
"Attendance": [85, 90, 88, 75, 80],
"Grade": ["B", "A", "A", "C", "B"]}.
b. Display Dataset Information
c. Display Statistical Summary
d. Display only student names and marks
e. Display students scoring more than 80 marks.
f. Sort students based on marks in descending order.
g. Add a new column named “Bonus” containing: 5 marks bonus for every student.

PROGRAM:

```
import pandas as pd
data = {"Name": ["Aman", "Diya", "Rahul", "Sneha", "Arjun"],
   "Department": ["CS", "IT", "CS", "ECE", "IT"],
   "Marks": [78, 92, 85, 67, 74],
   "Attendance": [85, 90, 88, 75, 80],
   "Grade": ["B", "A", "A", "C", "B"]}
df = pd.DataFrame(data)
print("DATASET:")
print(df)
print("\nDATASET INFORMATION:")
print(df.info())
print("\nSTATISTICAL SUMMARY:")
print(df.describe())
print("\nSTUDENT NAMES AND MARKS:")
print(df[["Name", "Marks"]])
print("\nSTUDENTS SCORING MORE THAN 80 MARKS:")
print(df[df["Marks"] > 80])
print("\nSTUDENTS SORTED BY MARKS (DESCENDING):")
print(df.sort_values(by="Marks", ascending=False))
df["Bonus"] = 5
print("\nDATASET AFTER ADDING BONUS:")
print(df)
```

OUTPUT:


HUSNA LAB CYCLE
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
[23]
print("\nSTUDENTS SORTED BY MARKS (DESCENDING):")
print(df.sort_values(by="Marks", ascending=False))
df["Bonus"] = 5
print("\nDATASET AFTER ADDING BONUS:")
print(df)
DATASET:
Name Department Marks Attendance Grade
0 Aman CS 78 85 B
1 Diya IT 92 90 A
2 Rahul CS 85 88 A
3 Sneha ECE 67 75 C
4 Arjun IT 74 80 B
DATASET INFORMATION:
Name Department Marks Attendance Grade
0 Aman CS 78 85 B
1 Diya IT 92 90 A
2 Rahul CS 85 88 A
3 Sneha ECE 67 75 C
4 Arjun IT 74 80 B
STATISTICAL SUMMARY:
Marks Attendance
count 5.000000 5.000000
mean 79.200000 83.600000
std 9.679876 6.107373
min 67.000000 75.000000
25% 74.000000 80.000000
50% 78.000000 85.000000
75% 85.000000 88.000000
max 92.000000 90.000000

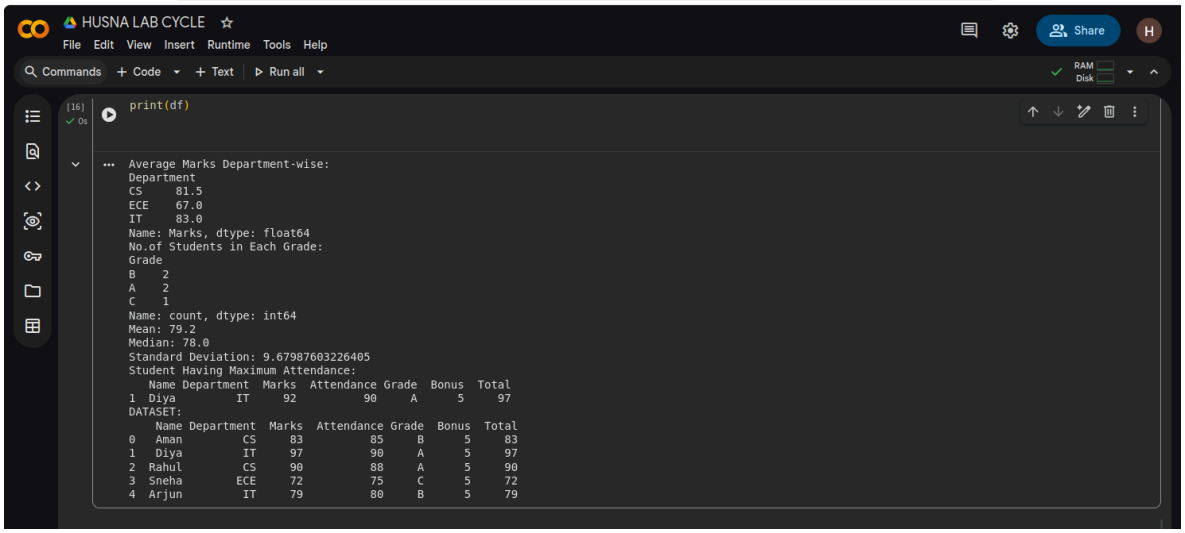

HUSNA LAB CYCLE
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
max 92.000000 90.000000
STUDENT NAMES AND MARKS:
Name Marks
0 Aman 78
1 Diya 92
2 Rahul 85
3 Sneha 67
4 Arjun 74
STUDENTS SCORING MORE THAN 80 MARKS:
Name Department Marks Attendance Grade
1 Diya IT 92 90 A
2 Rahul CS 85 88 A
STUDENTS SORTED BY MARKS (DESCENDING):
Name Department Marks Attendance Grade
1 Diya IT 92 90 A
2 Rahul CS 85 88 A
0 Aman CS 78 85 B
4 Arjun IT 74 80 B
3 Sneha ECE 67 75 C
DATASET AFTER ADDING BONUS:
Name Department Marks Attendance Grade Bonus
0 Aman CS 78 85 B 5
1 Diya IT 92 90 A 5
2 Rahul CS 85 88 A 5
3 Sneha ECE 67 75 C 5
4 Arjun IT 74 80 B 5

13) Using the above dataframe answer the following questions.
a. Create a new column named “Total” by adding marks and bonus.
b. Find average marks department-wise.
c. Count the number of students in each grade category.
d. Calculate: mean, median,standard deviation of student marks.
e. Display students having maximum attendance.
f. Increase marks of all students by 5.

PROGRAM:

```
import pandas as pd
data = {"Name":["Aman", "Diya", "Rahul", "Sneha", "Arjun"],
        "Department":["CS", "IT", "CS", "ECE", "IT"],
        "Marks": [78, 92, 85, 67, 74],
        "Attendance":[85, 90, 88, 75, 80],
        "Grade": ["B", "A", "A", "C", "B"]}
df = pd.DataFrame(data)
df["Bonus"] = 5
df["Total"] = df["Marks"] + df["Bonus"]
print("Average Marks Department-wise:")
print(df.groupby("Department")["Marks"].mean())
print("No.of Students in Each Grade:")
print(df["Grade"].value_counts())
print("Mean:", df["Marks"].mean())
print("Median:", df["Marks"].median())
print("Standard Deviation:", df["Marks"].std())
max_attendance = df["Attendance"].max()
print("Student Having Maximum Attendance:")
print(df[df["Attendance"] == max_attendance])
df["Marks"] = df["Marks"] + 5
print("DATASET:")
print(df)
```

OUTPUT:


HUSNA LAB CYCLE
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
[16]
0s
print(df)
Average Marks Department-wise:
Department
CS 81.5
ECE 67.0
IT 83.0
Name: Marks, dtype: float64
No.of Students in Each Grade:
Grade
B 2
A 2
C 1
Name: count, dtype: int64
Mean: 79.2
Median: 78.0
Standard Deviation: 9.67987603226405
Student Having Maximum Attendance:
Name Department Marks Attendance Grade Bonus Total
1 Diya IT 92 90 A 5 97
DATASET:
Name Department Marks Attendance Grade Bonus Total
0 Aman CS 83 85 B 5 83
1 Diya IT 97 90 A 5 97
2 Rahul CS 90 88 A 5 90
3 Sneha ECE 72 75 C 5 72
4 Arjun IT 79 80 B 5 79

14) Write a Python program using Pandas to clean and preprocess the following student dataset.

a. Perform the following operations:

i. Create a DataFrame containing student details.
ii. Identify missing values in the dataset.
iii. Replace missing values in the Marks column using the mean value.
iv. Remove duplicate rows from the dataset.
v. Convert student names into uppercase.
vi. Rename the column Marks to Score.
vii. Display the cleaned dataset.

PROGRAM:

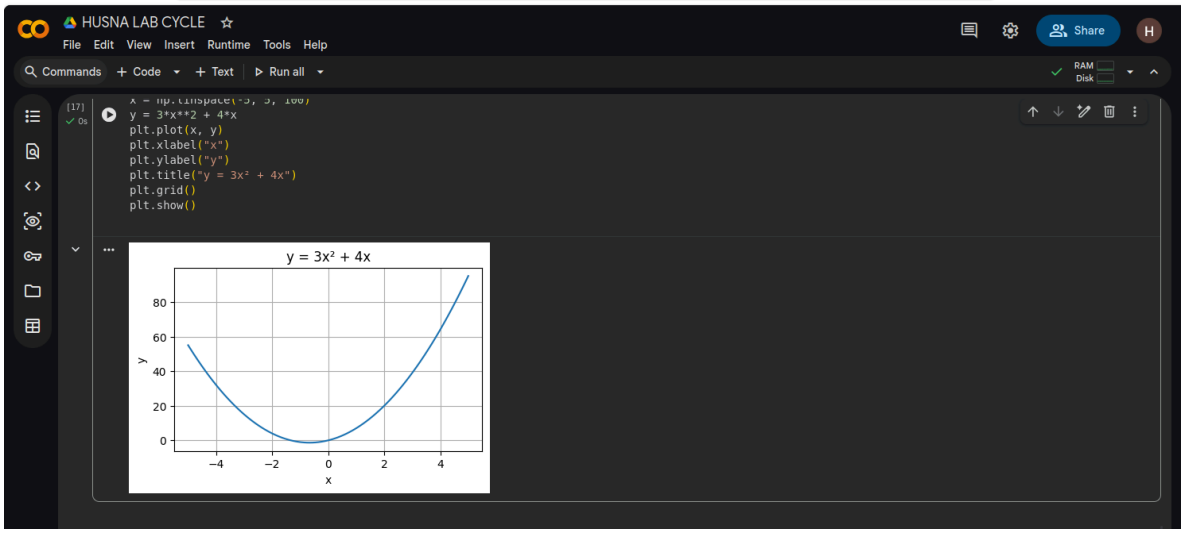
OUTPUT:

15) Plot simple mathematical functions.[y=3x^2+4x]

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
plt.figure(figsize=(5, 3))
x = np.linspace(-5, 5, 100)
y = 3*x**2 + 4*x
plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.title("y = 3x² + 4x")
plt.grid()
plt.show()
```

OUTPUT:

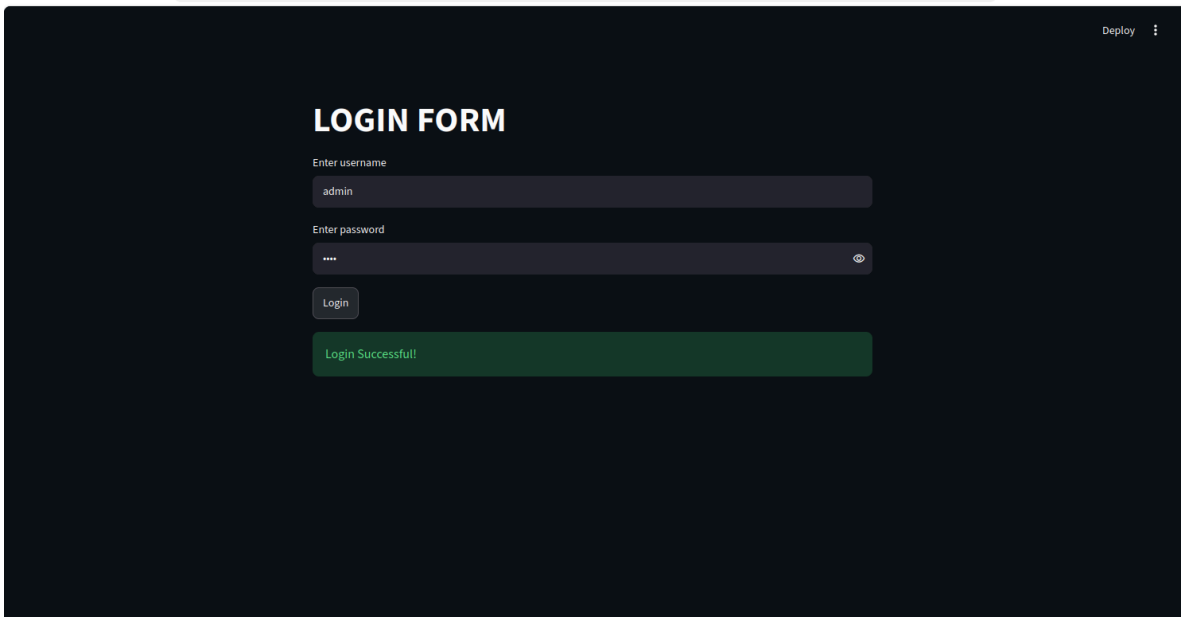


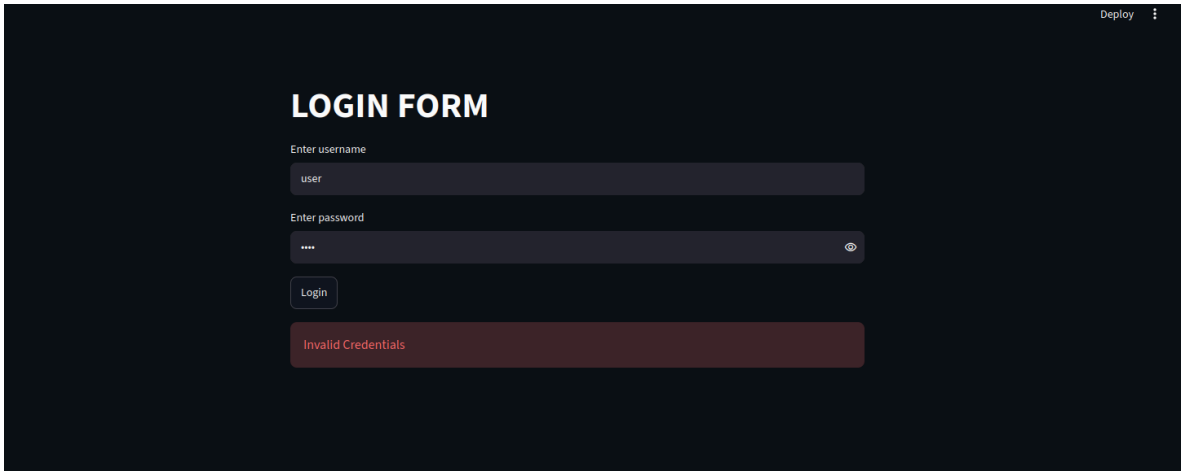
16) Create simple login form using streamlit

PROGRAM:

```
%%writefile DEMO-GUI.py
import streamlit as st
st.title("LOGIN FORM")
username=st.text_input("Enter username")
password=st.text_input("Enter password",type="password")
if st.button("Login"):
    if username=="admin" and password=="1234":
        st.success("Login Successful!")
    else:
        st.error("Invalid Credentials")
```

OUTPUT:


Deploy
LOGIN FORM
Enter username
admin
Enter password
Login
Login Successful!

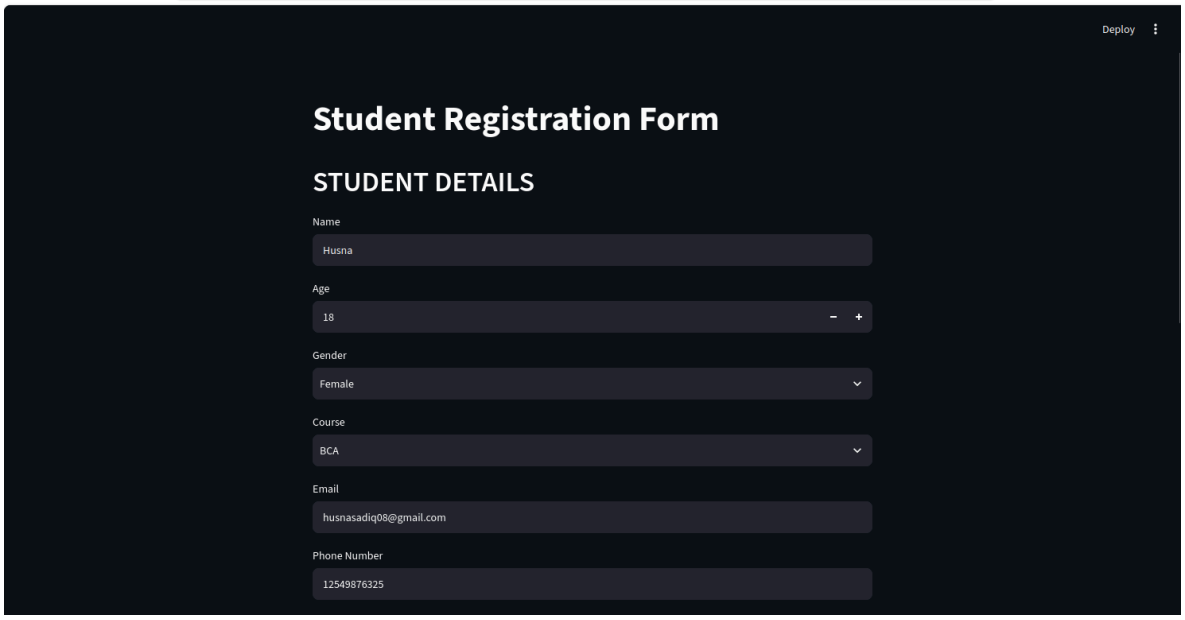

Deploy
LOGIN FORM
Enter username
user
Enter password
Login
Invalid Credentials

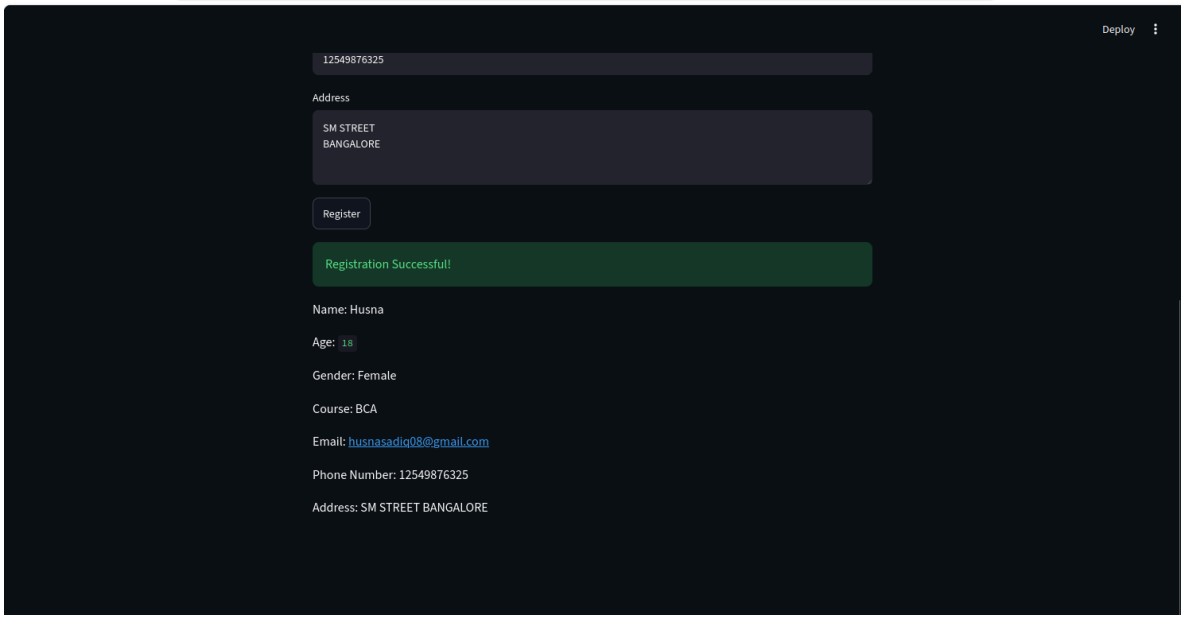
17) Create student registration form using streamlit

PROGRAM:

```
%%writefile DEMO-GUI.py
import streamlit as st
st.title("Student Registration Form")
st.header("STUDENT DETAILS")
name=st.text_input("Name")
age=st.number_input("Age",min_value=1, max_value=100)
gender=st.selectbox("Gender",["Male","Female","Other"])
course=st.selectbox("Course",["BCA","BBA","BA","BSc","Bcom"])
email=st.text_input("Email")
phone=st.text_input("Phone Number")
address=st.text_area("Address")
if st.button("Register"):
    st.success("Registration Successful!")
    st.write("Name:",name)
    st.write("Age:",age)
    st.write("Gender:",gender)
    st.write("Course:",course)
    st.write("Email:",email)
    st.write("Phone Number:",phone)
    st.write("Address:",address)
```

OUTPUT:


Deploy
Student Registration Form
STUDENT DETAILS
Name
Husna
Age
18
Gender
Female
Course
BCA
Email
husnasadiq08@gmail.com
Phone Number
12549876325


Deploy
12549876325
Address
SM STREET
BANGALORE
Register
Registration Successful!
Name: Husna
Age: 18
Gender: Female
Course: BCA
Email: husnasadiq08@gmail.com
Phone Number: 12549876325
Address: SM STREET BANGALORE

18) Sample program to use Git and Github.

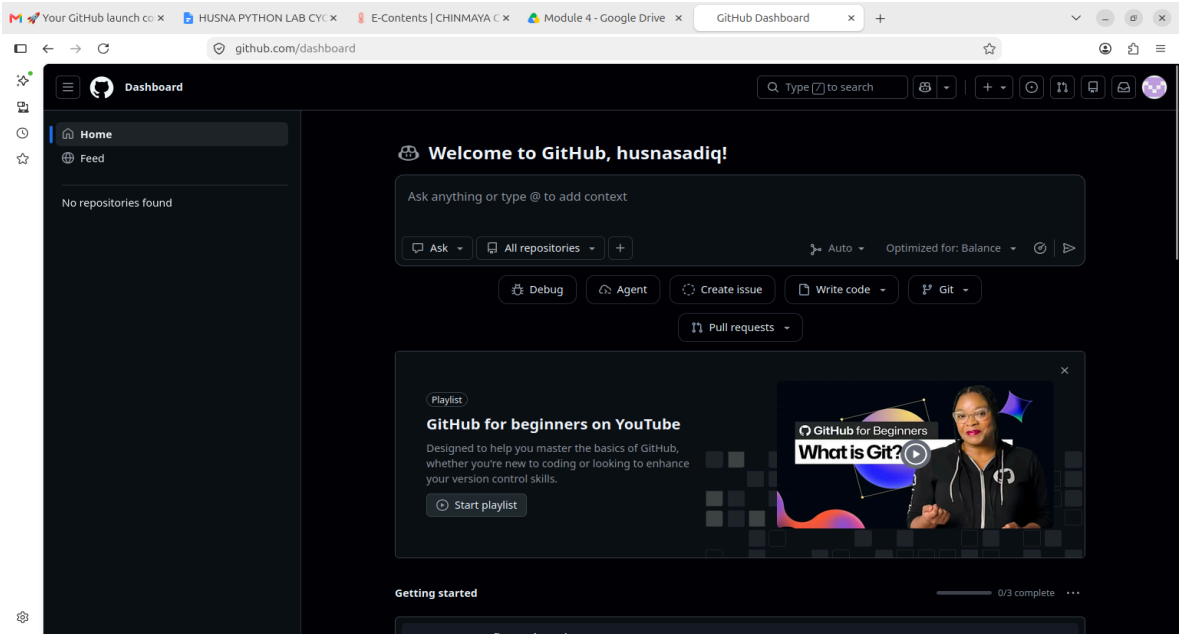
PROGRAM:

Install Git in Ubuntu

$sudo apt update
$sudo apt upgrade

$git --version

Create a GitHub Account



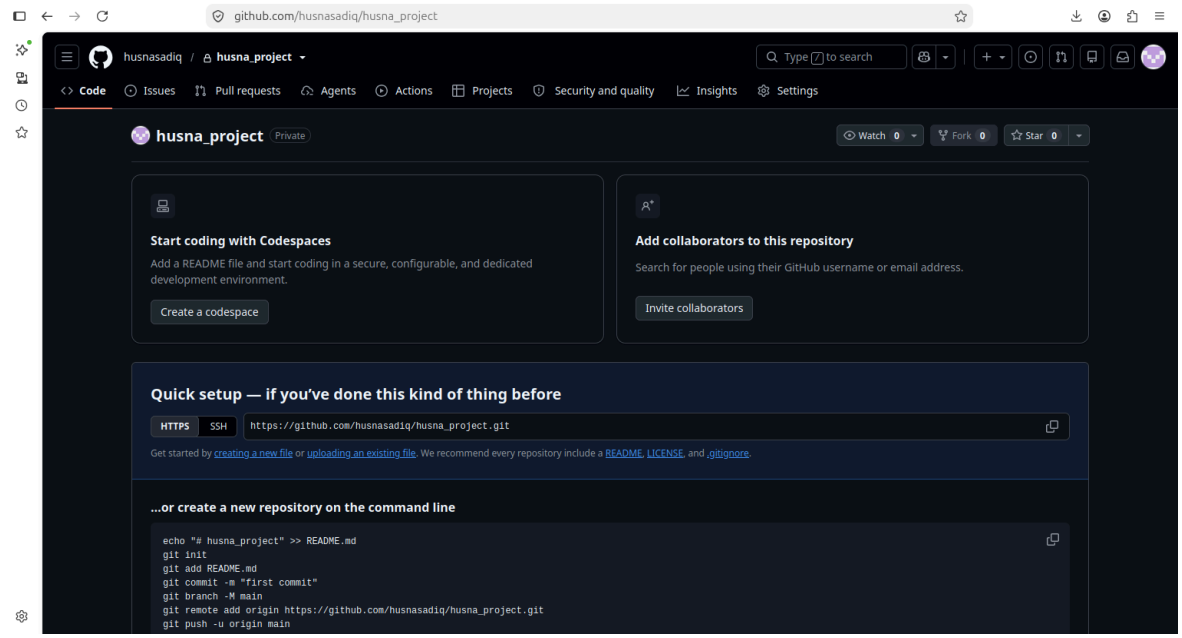
Set your username
$git config --global user.name "Your Name"

Set your email
$git config --global user.email "your_email@example.com"

```
bca2@chinarts-ThinkCentre-neo-50q-Gen-4:~$ git config --global user.name"husnasadiq-git"
bca2@chinarts-ThinkCentre-neo-50q-Gen-4:~$ git config --global user.email"husna@sadiq.com"
```

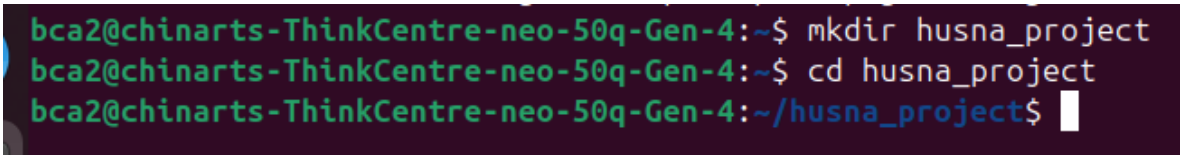
Check configuration
$git config --global --list

Create a Repository on GitHub
1. Sign in.
2. Click + at the top-right.
3. Select New repository.
4. Enter repository name: student_project
5. Choose Public or Private.
6. Click Create repository.



Create and Go to your project folder

$mkdir student_project
$cd student_project



Create a Python file in the project folder: $vi hello.py

```
bca2@chinarts-ThinkCentre-neo-50q-Gen-4:~/husna_project$ vi hello.py
bca2@chinarts-ThinkCentre-neo-50q-Gen-4:~/husna_project$ python3 hello.py
Welcome to Git and GitHub
bca2@chinarts-ThinkCentre-neo-50q-Gen-4:~/husna_project$
```

